

# DevNet Resource Validation

CS0016, Network and Communications 2 | Module 2, The DevNet Developer Environment

|                 |                                      |                     |                    |
|-----------------|--------------------------------------|---------------------|--------------------|
| <b>Name</b>     | Dawn Andrei C. Pamesa                | <b>Section</b>      | TS31               |
| <b>Activity</b> | Module 2, DevNet Resource Validation | <b>Date</b>         | September 13, 2026 |
| <b>Mode</b>     | Individual                           | <b>AI tool used</b> | Claude (Anthropic) |

## 1. Purpose

Choosing the right DevNet resource before starting a network-automation task matters because each resource answers a different question, and picking the wrong one wastes the limited time a task usually has. An Always-On Sandbox that is grabbed for a configuration test will block the change or spill it onto other users, while a Reservation Sandbox booked for a simple read-only API call means waiting for provisioning that was never needed in the first place. Selecting the resource first, and confirming it against official Cisco documentation, keeps the environment matched to the actual requirement instead of to whichever option was fastest to reach.

## 2. Validated Resource Decisions

All four selections below were checked against pages on [developer.cisco.com](https://developer.cisco.com) before being written into `student_plan.json`.

| Use case                               | Selected resource   | Decisive requirement                                                                                                                                     | Official Cisco evidence                                                                   | Status             |
|----------------------------------------|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|--------------------|
| UC1<br>Quick read-only API exploration | always-on-sandbox   | The team cannot wait for provisioning, and only needs safe read-only requests with no administrative changes.                                            | <a href="https://developer.cisco.com/docs/sandbox/">developer.cisco.com/docs/sandbox/</a> | verified           |
| UC2<br>Private configuration testing   | reservation-sandbox | Administrative control inside a private environment, with scheduling, VPN, and setup time already accepted by the team.                                  | <a href="https://developer.cisco.com/docs/sandbox/">developer.cisco.com/docs/sandbox/</a> | partially-verified |
| UC3<br>Guided API concept practice     | learning-lab        | Structured, step-by-step content is the immediate need, and the goal is guided practice rather than access to administrative devices.                    | <a href="https://developer.cisco.com/learning/">developer.cisco.com/learning/</a>         | partially-verified |
| UC4<br>Reusable automation example     | code-exchange       | Existing community and Cisco-maintained repositories have to be examined before a new solution is designed, so the need is source code and not a device. | <a href="https://developer.cisco.com/codeexchange/">developer.cisco.com/codeexchange/</a> | rejected           |

Both sandbox decisions cite the same documentation page on purpose, because that page is where Cisco documents the Always-On and Reservation access models side by side, and the decision between UC1 and UC2 is precisely a decision between those two models.

### 3. AI Evaluation

#### *Rejected, UC4*

The AI recommended `always-on-sandbox` and argued it clearly, saying the developer could start immediately and run the automation code against real devices without a booking. The reasoning was internally consistent, however it answered a requirement the scenario never made. UC4 asks for community and Cisco-maintained repositories to be examined before a new solution is designed, which is a request for existing code and not for somewhere to execute it. The Cisco Code Exchange page documents it as the curated index of exactly those repositories, so I discarded the recommendation and selected `code-exchange` instead. A sandbox would only matter later, after a repository is picked and the code has to be reviewed and tested.

#### *Modified, UC3*

The AI chose `learning-lab`, which was right, but it also claimed the labs give the learner their own admin-level devices that remain available after the lab finishes. I could not support that claim, and it contradicts the module's own decision guide, which states that a Learning Lab is learning content and not a substitute for choosing an execution environment. I kept the resource type and recorded the status as `partially-verified` rather than accepting the full answer just because the label was correct.

#### *Modified, UC2*

The AI chose `reservation-sandbox` correctly, but stated that a reservation always grants full administrator rights on every device in the pod. Privilege detail is documented per individual sandbox rather than as one blanket guarantee, so the status was recorded as `partially-verified`.

#### *Accepted, UC1*

Every claim the AI raised, meaning no reservation needed, shared rather than isolated, and limited administrative privileges, matched the documented Always-On access model, so this one was accepted as verified.

The pattern I noticed across all four is that the AI was most convincing exactly where it was wrong, since the UC4 answer was the best-written one of the set and still solved the wrong problem.

### 4. Validation Evidence

- **Validator result:** VALIDATION COMPLETE: 9/9 checks passed.
- **Command used:** `python validate_plan.py`
- **Official Cisco pages reviewed:** DevNet Sandbox documentation ([developer.cisco.com/docs/sandbox/](https://developer.cisco.com/docs/sandbox/)), DevNet Learning Labs ([developer.cisco.com/learning/](https://developer.cisco.com/learning/)), and Cisco Code Exchange ([developer.cisco.com/codeexchange/](https://developer.cisco.com/codeexchange/)).

No reservation was created, no VPN was connected, no API request was sent, and no sandbox was modified. All exploration stayed read-only, as required by Section 4 of the activity.

### ***Validator output***

```
$ python validate_plan.py
PASS: JSON file loaded
PASS: student and AI disclosure completed
PASS: all four scenario IDs present
PASS: resource classifications match scenario requirements
PASS: official Cisco evidence URLs supplied
PASS: AI verification statuses are valid
PASS: rationales are sufficiently detailed
PASS: AI recommendations are summarized in the student's own words
PASS: no credential-like fields detected
VALIDATION COMPLETE: 9/9 checks passed.
```

## **5. Git Evidence**

- **Initial commit message:** Set up Module 2 workspace with DevNet starter files and README skeleton
- **Validation commit message:** Validate all four DevNet resource decisions against Cisco docs, 9/9 checks passed

### ***Output of* git log --oneline**

```
5281d4c Record git log output as traceability evidence in README
a60491d Validate all four DevNet resource decisions against Cisco docs, 9/9 checks passed
64b86d9 Set up Module 2 workspace with DevNet starter files and README skeleton
```

## **6. Reflection Questions**

### **1. Which scenario was easiest to classify, and which requirement determined your choice?**

UC1 was the easiest one to classify. The scenario already gives away the answer in its last line, which is that the team cannot wait for provisioning. Once that requirement is spotted, the Reservation Sandbox is eliminated immediately, because reserving a sandbox is a waiting step by definition. Learning Labs and Code Exchange were never really in the running either, since the team wants to send actual API requests and not read a tutorial or browse repositories. That leaves the Always-On Sandbox, and the rest of the scenario supports it too, because shared access and read-only requests are exactly the conditions an Always-On environment is built for.

## **2. Describe one AI claim that needed correction, qualification, or rejection.**

The clearest one was UC4. The AI recommended always-on-sandbox and explained it well, saying that the developer could start immediately and run the automation code against real devices. The reasoning sounded fine on its own, however it answered a requirement that the scenario never stated. The scenario asks for community and Cisco-maintained repositories to examine before designing a new solution, so what is needed is existing code, not a device to run it on. I rejected the recommendation and selected code-exchange instead, based on the Code Exchange page describing it as a curated index of those repositories.

A second one worth noting is UC3, where the AI got the resource right but attached a claim that Learning Labs give the learner admin-level devices to keep afterwards. I could not support that, and it also runs against the module's own warning that a Learning Lab is learning content and not a substitute for an execution environment, so I recorded it as partially-verified rather than just accepting the whole answer because the label happened to be correct.

## **3. Why is an Always-On Sandbox not automatically the best option for every API task?**

Because the things that make it convenient are the same things that limit it. An Always-On Sandbox is shared and immediately reachable, which is great when the task is read-only exploration, yet the sharing is precisely the problem once the task involves configuration. It is a shared environment with restricted administrative access, so a configuration change would either be blocked outright or, worse, would land on an environment other people are using at the same time. There is also no isolation to speak of, which means a test cannot be assumed to be clean or repeatable, since somebody else may be working in the same place. For anything that needs administrative control, a private environment, or a predictable starting state, the Reservation Sandbox is the correct choice even though it costs scheduling and setup time.

I realized while doing this that “fastest to access” and “most appropriate” are two different questions, and the second one is the one being graded.

## **4. How did Git improve the traceability of your resource-selection process?**

Git made the process reviewable instead of just the final answer being reviewable. The first commit captured the workspace before any decision existed, and the validation commit captured the finished plan together with the validator output, so the two states can be compared and it is visible what was actually decided at which point rather than only what the file looks like now. Because the prompt record and the verification statuses are committed alongside the plan, anyone checking the work can see which AI claims were accepted, which were qualified, and which one was thrown out, and they can see that the rejection happened before the plan was validated and not afterwards as a cleanup.

To conclude, the commits turn the plan from a single submitted file into a record that shows the order the work was done in, which is the part that matters when the requirement is to prove that an AI recommendation was verified and not just copied.

## 7. AI-Use Disclosure

**Tool used.** Claude (Anthropic).

**Type of assistance received.** The AI was given one prompt per scenario, following the prompt pattern in Section 8 of the activity sheet. Each prompt asked for exactly one resource type out of the four allowed labels, the requirement that drove the choice, and any access, isolation, setup, or privilege claim that I should verify. The full prompts and the reply excerpts are recorded in `ai_prompt_record.md` inside the submitted repository.

**What was independently checked.** Every recommendation was treated as provisional until an official page on `developer.cisco.com` supported it. I opened the DevNet Sandbox documentation for UC1 and UC2, the Learning Labs page for UC3, and the Code Exchange page for UC4, and I compared each AI claim against the resource description, the access model, and the intended use before recording a status.

**What was revised.** Three of the four answers were changed in some way. UC4 was rejected outright and replaced with code-exchange. UC3 and UC2 kept the recommended resource type but had unsupported privilege claims stripped out, and both were recorded as partially-verified instead of verified. Only UC1 survived unchanged.

**What was not shared with the AI.** No usernames, passwords, keys, tokens, reservation emails, private repository content, or personal data were pasted into the tool. Only the fictional scenarios supplied in `devnet_use_cases.json` were used.

## 8. Files Submitted

| File                                                    | Purpose                                                                                                                                               |
|---------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>Pamesa_DawnAndrei_CS0016_M2_DevNetPlan.zip</code> | The local Git repository with all four validated decisions, the README, the AI prompt record, the reflection answers, and the saved validator output. |
| <code>Pamesa_DawnAndrei_CS0016_M2_DevNetPlan.pdf</code> | This document.                                                                                                                                        |
| <code>student_plan.json</code>                          | The completed plan, inside the zip.                                                                                                                   |
| <code>README.md</code>                                  | Decisions, official evidence, Git record, and AI disclosure, inside the zip.                                                                          |
| <code>ai_prompt_record.md</code>                        | Prompt and reply excerpt for each of the four scenarios.                                                                                              |
| <code>reflection.md</code>                              | The Section 11 reflection answers, inside the zip.                                                                                                    |
| <code>validator_output.txt</code>                       | Saved validator output showing 9/9 checks passed.                                                                                                     |

Note on the course code: the activity handout is written against the generic placeholder code IT0123. The live class code is CS0016, Network and Communications 2, so this document and the submitted repository use CS0016. The starter file `devnet_use_cases.json` is left exactly as it was supplied, including its internal IT0123 course string, so that the provided files remain unmodified.